Programa: CI DIgital

Fase: Residência Tecnológica

Equipe:

Felipe Guedes de Oliveira

Matheus Grossi

Vicente José Campitelli Real

Projeto: CNN em Hardware

Campinas/SP

Outubro.2026

SUMÁRIO

1. Descrição do Problema **4**
1.1. CNN (Convolutional Neural Network - Rede Neural Convolucional) 4
2. Objetivos **5**
3. Atividades Desenvolvidas **7**
4. Cronograma **7**
5. Arquitetura do Projeto **8**
5.1. Organização do repositório 8
Principais diretórios: 8
Organização completa do repositório: 9
5.2. Módulos RTL (Diretório src/) 10
5.3. Ambiente de Verificação (Diretório UVM/) 11
5.4. Rascunho inicial da arquitetura 11
5.5. Notas do bloco CNN e materiais da documentação 12
5.6. Bloco DotProduct 13
5.7. Bloco Convolution 14
5.8. Bloco FeatureMap e ConvLayer 14
5.9. Conceitos aplicados ao projeto 15
6. Síntese **16**
6.1. Constraints do projeto: 16
6.2. Visão Geral do Layout da implementação 16
Visão geral do layout e da implementação 18
Posicionamento e roteamento 19
Análise de congestionamento e potência 21
Relatórios de clock e timing 22
6.3. Contagem de Células (Cell Count) 23
6.4. Análise de potência e report_power 25
6.5. Trecho de STA do OpenROAD 27
6.6. Relatórios de CTS e layout associado 28
7. Verificação **30**
7.1. Estratégia de estímulos 30
7.2. Waveforms da simulação 31
7.3. Relatório da taxa de cobertura da verificação 34
7.4. Análise de resultados 34
7.5. Casos de falha/Incomplete 34
7.6. Análise final da bancada 35
8. Limitações no desenvolvimento e solução de contorno adotadas **36**
9. Scripts **37**
10. Conclusão **37**
11. Referências **37**
12. Anexos **38**
Explicação completa CNN.sv 38
Todos os parâmetros explicados 38

Parâmetros locais 39
INPUTS e OUTPUTS 40
Sinais lógicos intermediários 41
Blocos declarados 41

1. Descrição do Problema

Atualmente, a integração de Inteligência Artificial em dispositivos de ponta (como câmeras inteligentes, drones e dispositivos médicos) enfrenta o desafio da “dependência da nuvem”. Processar algoritmos de visão computacional em servidores externos gera custos elevados de infraestrutura, latência na resposta e riscos de privacidade, o que exige que parte do processamento seja realizado na borda. Este projeto transfere o peso do processamento para um hardware dedicado. Isso permite tomada de decisões locais, reduzindo latência, necessidade de largura de banda e custos com servidores.

1.1. *CNN* (*Convolutional Neural Network* - Rede Neural Convolucional)

Em Inteligência Artificial e Processamento Digital de Sinais, refere-se às redes neurais profundas focadas em visão computacional. Seu princípio de funcionamento é baseado na extração de características importantes da imagem através da convolução com kernels, em múltiplas camadas, e passando, por fim, por camadas de neurônios em MLP simples (*fully connected)*, para obter a probabilidade final das classes que o conjunto de imagens pode tomar.

A solução proposta é a implementação de uma rede CNN em ***Verilog/SystemVerilog***, voltada para processamento de imagens, extração de características e classificação por meio de camadas convolucionais, de pooling, flatten e totalmente conectadas.

2. Objetivos

O objetivo principal é projetar uma arquitetura de hardware otimizada para a CNN, para inferência em tempo real realizada em dispositivo local.

Os principais aspectos considerados são:

- **Matriz de Processamento:** Desenvolver um conjunto de elementos de processamento (PEs) paralelizados, capaz de computar janelas de convolução simultaneamente.
- **Verificação Robusta:** Construir um ambiente de verificação avançado para garantir cobertura funcional completa e ausência de bugs sob condições de irradiância variáveis.
- **Síntese Lógica:** Realizar a síntese do design gerando um netlist otimizado e extraindo métricas precisas de potência, área e temporização (timing) que comprovem a viabilidade técnica da solução em silício.

A arquitetura proposta foi organizada para suportar simulação funcional, validação e síntese, além de registrar relatórios e resultados obtidos em diferentes etapas do fluxo de projeto.

A CNN implementada processa imagens de entrada, aplicando uma sequência de operações clássicas de redes neurais:

- convolução para extração de características;
- quantização para representar valores em ponto fixo;
- ativação ReLU para introduzir não-linearidade;
- max pooling para reduzir dimensionalidade;
- flatten para transformar mapas de características em vetor;
- camadas totalmente conectadas para gerar a saída final da rede.

A estrutura principal do módulo de topologia está em `*src/CNN.sv*`, onde a rede é montada com duas camadas convolucionais, operações de pooling, flatten e duas camadas fully connected. O fluxo principal da arquitetura segue a sequência abaixo:

1. entrada da imagem;
2. primeira camada convolucional;
3. quantização e *ReLU*;
4. max pooling;
5. segunda camada convolucional;
6. quantização e *ReLU*;
7. *max pooling*;
8. flatten;
9. camada *fully connected 1*;
10. camada *fully connected 2*.

O módulo final produz um vetor de saídas correspondente às classes ou valores de classificação do sistema.

3. Atividades Desenvolvidas

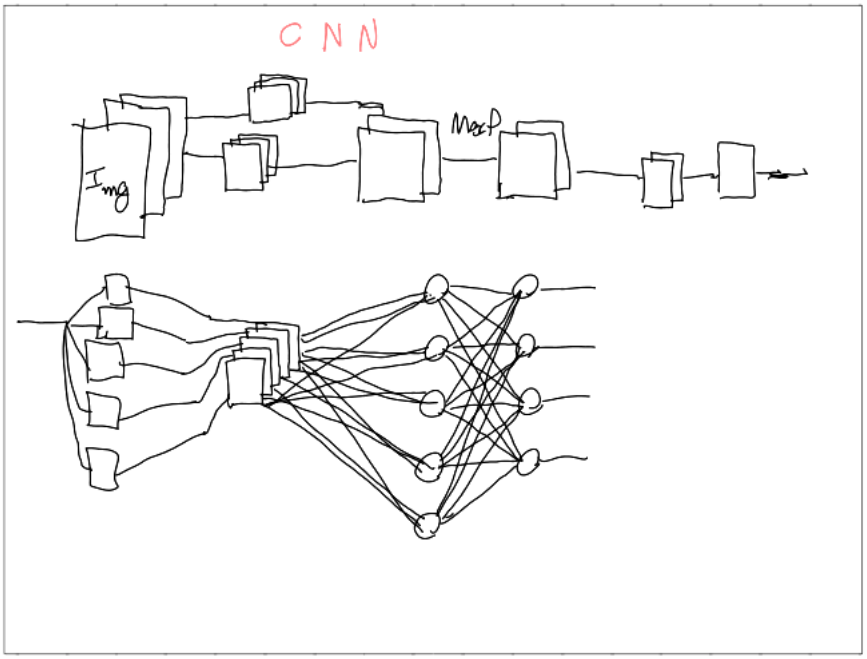
- **Modelagem e Especificação:** Definição da arquitetura de *hardware* e modelagem matemática da arquitetura CNN do bloco.
- **Design RTL:** Codificação dos blocos anteriormente explicados da CNN em *Verilog / SystemVerilog*.
- **Desenvolvimento do Ambiente de Verificação:** Criação de um ambiente de verificação utilizando a metodologia *UVM* para uma verificação automatizada.
- **Verificação Funcional:** Execução de testes para validar a lógica sob diferentes perfis de carga e irradiância e análise de métricas de cobertura a fim de garantir que todos os estados do algoritmo e cenários de irradiância foram validados.
- **Síntese Lógica:** Mapeamento do design para uma biblioteca de células padrão.
- **Análise de Métricas (PPA):** Extração e avaliação de relatórios de Potência, Performance (*Timing*) e Área para validação dos requisitos de silício.

4. Cronograma

- **40h - Modelagem e Documentação:** Pré-estudo, dedução matemática em ponto fixo e definição da máquina de estados. Entrega: Especificação e Plano de verificação.
- **80h - Codificação RTL e Integração de Blocos:** Entrega: RTL em Verilog/SystemVerilog dos sub-blocos e bloco de topo.
- **80h - Desenvolvimento do Ambiente de Verificação:** Entrega: Ambiente de verificação UVM estável e pronto para regressão.
- **100h - Regressão Funcional:** Entrega: Relatório de Testes, Bugs e Cobertura Final.
- **50h - Síntese Lógica:** Entrega: Netlist, SDF, geração de relatórios de área e consumo de potência.
- **50h - Verificação a nível de portas lógicas:** Entrega: Relatório de Verificação da Netlist.

5. Arquitetura do Projeto

De modo geral, um bloco CNN pode ser representado pela seguinte arquitetura:



A imagem resume a ideia principal da rede: a entrada da imagem passa por uma sequência de blocos de convolução, extração de características, *pooling* e classificação final, com o processamento distribuído por camadas que evoluem em abstração e complexidade.

5.1. Organização do repositório

Principais diretórios:

- `src/`: módulos principais da CNN, incluindo camadas convolucionais, *pooling*, *flatten*, quantização, *ReLU* e capacidade de produção do *output* final.
- `synth_org/`: estrutura organizada para síntese lógica e relatórios de implementação, incluindo diretórios por bloco funcional e por topologia do projeto.
- `scripts/`: automações, arquivos de suporte, makefiles e rotinas para síntese e testes.
- `testbenchs/`: ambientes de simulação para verificar componentes específicos da arquitetura.
- `UVM/`: infraestrutura de verificação em *UVM* para validação comportamental e regressão.
- `results_tb/` e `results_uvm/`: resultados de simulação, logs, relatórios de cobertura e artefatos gerados.
- `docs/` e `images/`: materiais documentais e recursos visuais do projeto.

Organização completa do repositório:

```
CNN
├── README.md
├── script.bat
├── docs/
├── images/
├── oneware_project_folder/
│   └── CNN/
│       └── CNN.fpgaproj
├── src/
│   ├── CNN.sv
│   ├── ConvLayer.sv
│   ├── Convolution.sv
│   ├── FCLayer.sv
│   ├── FeatureMap.sv
│   ├── Flatten.sv
│   ├── MaxPooling.sv
│   ├── Neuron.sv
│   ├── Quantizer.sv
│   ├── logic_filelist.txt
│   ├── DotProduct/
│   ├── Multiplier&Adder/
│   └── ReLU/
├── synth_org/
│   ├── README.md
│   ├── logic_filelist.txt
│   ├── arithmetic/
│   ├── convolution/
│   ├── fully_connected/
│   ├── pooling/
│   ├── reshape/
│   └── top/
├── scripts/
│   ├── synth/
│   ├── testbenchs/
│   ├── bash/
│   ├── makefiles/
│   └── UVM/
├── testbenchs/
```

```
│   ├── arithmetic/
│   ├── DOT/
│   └── ReLU/
├── UVM/
│   ├── DOT/
│   └── ReLU/
├── results_tb/
│   ├── DOT/
│   ├── ReLU/
│   └── coverage/
├── results_uvm/
│   ├── DOT/
│   ├── ReLU/
│   ├── coverage/
│   └── logs/
└── out/
```

5.2. Módulos RTL (Diretório *src/*)

A implementação do hardware está dividida nos seguintes módulos principais:

- *CNN.sv*: Módulo de topo que integra todos os componentes da rede neural.
- *ConvLayer.sv*: Implementa a camada de convolução.
- *Convolution.sv*: Módulo responsável pela operação de convolução.
- *DotProduct.sv*: Implementa a operação de produto interno, utilizada tanto pela convolução quanto pela camada de neurônios.
- *FCLayer.sv*: Implementa a camada totalmente conectada (*Fully Connected*).
- *FeatureMap.sv*: Gerencia os mapas de características.
- *Flatten.sv:* Transforma a matriz multidimensional de características em um vetor de uma dimensão.
- *MaxPooling.sv*: Reduz a dimensionalidade da imagem, selecionando o maior valor dentre 4 *pixels*.
- *Neuron.sv*: Implementa um neurônio, recebendo entradas (incluindo o bias) que multiplicam por pesos, somando, e aplicando uma função de ativação na saída.
- *Quantizer.sv*: Converte a escala de saída de volta para a escala de entrada.
- *ReLU*.sv: Contém a implementação da função de ativação *ReLU*.

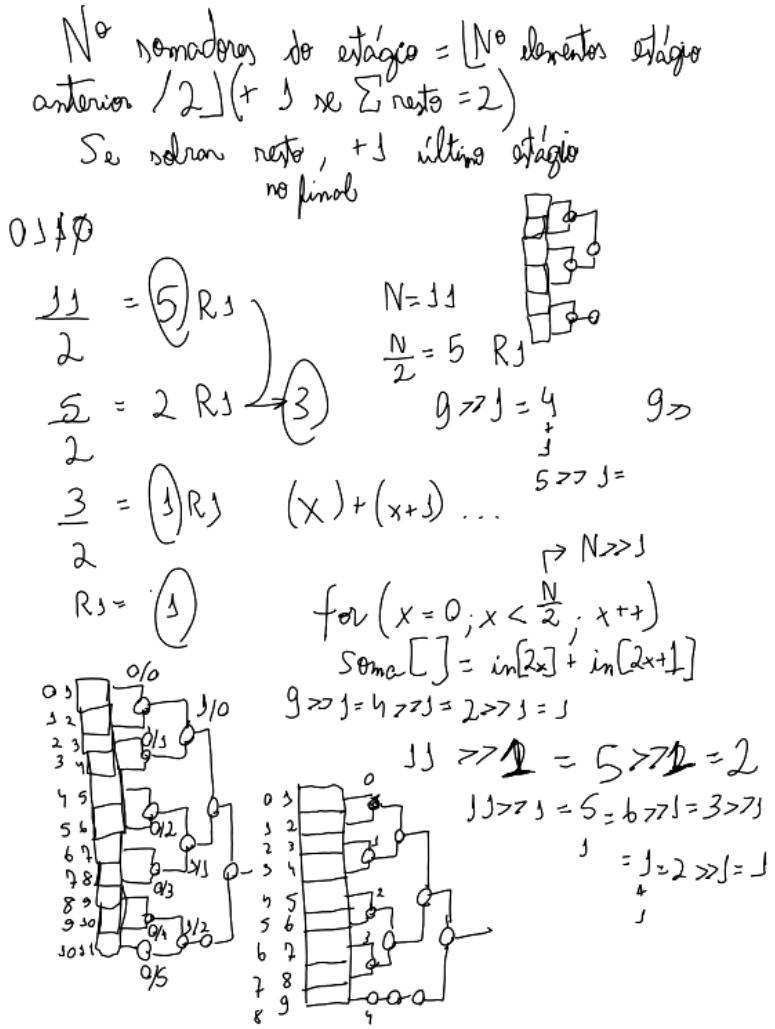
5.3. Ambiente de Verificação (Diretório UVM/)

O ambiente de verificação foi desenvolvido utilizando a metodologia *UVM* (*Universal Verification Methodology*), garantindo uma verificação automatizada e reutilizável. A estrutura atual inclui:

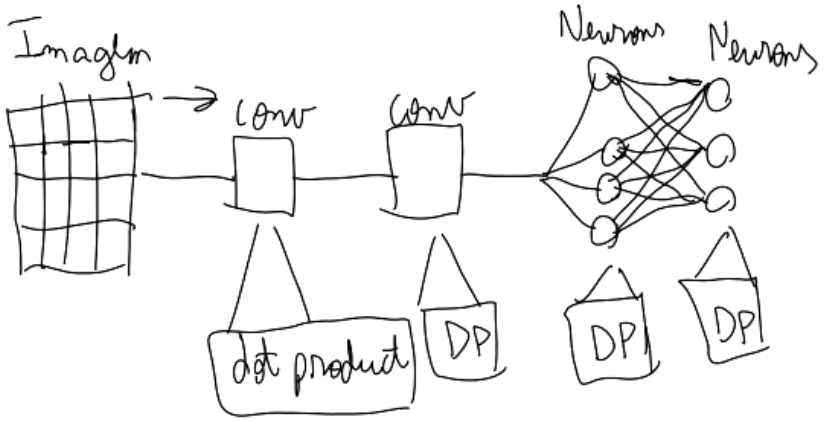
- *DOT/*: Verificação do bloco de produto escalar.
- *ReLU/*: Verificação do bloco de ativação *ReLU*.
- *template/*: Template base para criação de novos ambientes de verificação.

5.4. Rascunho inicial da arquitetura

A imagem abaixo demonstra o processo de raciocínio acerca da lógica de árvore de soma parametrizável.



Essa composição permite que a rede realize processamento de imagem em uma escala reduzida, mantendo o comportamento de redes de classificação orientadas por convolução.



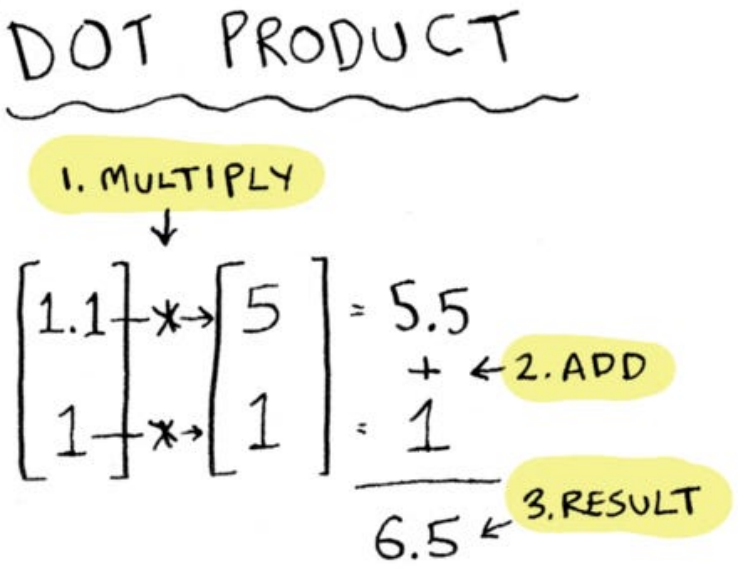
A figura acima representa o encadeamento conceitual da CNN: imagem de entrada, operação de convolução, processamento por blocos de *dot product* e neurônios, até a camada de decisão final. Essa visão complementa a descrição textual do fluxo da arquitetura e facilita a compreensão dos módulos implementados.

5.5. Notas do bloco *CNN* e materiais da documentação

Em complemento a esta documentação e ao README do projeto, na pasta docs, há um apanhado de materiais que documentam detalhes do projeto.

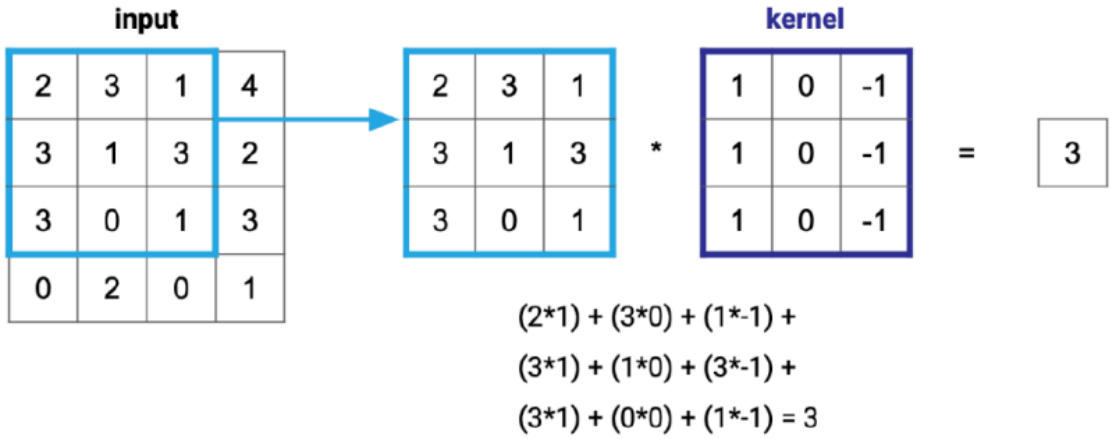
5.6. Bloco *DotProduct*

O bloco *DotProduct* é responsável pela multiplicação e soma entre dois vetores. De acordo com a documentação do projeto, "o produto interno entre o vetor [x_1, x_2, x_3] e [y_1, y_2, y_3] é igual a [(x_1 * y_1)+ (x_2 * y_2)+ (x_3 * y_3)]". Essa operação fundamenta tanto a convolução quanto as camadas totalmente conectadas.



5.7. Bloco *Convolution*

O bloco de convolução é uma operação em que um kernel é deslocado sobre uma imagem, produzindo uma nova matriz de saída, frequentemente chamada de *feature map*. A lógica do bloco permite que o kernel percorra a imagem, percorrendo trechos da matriz de entrada e gere os valores de saída correspondentes a cada janela de entrada.



Como pode ser observado na figura acima, cada valor da saída é obtido pela multiplicação dos elementos da janela com os pesos do kernel e pela soma dos produtos. Essa visualização representa o comportamento conceitual da operação aplicada e não um corte do RTL.

Esta operação pode ser interpretada, em muitos casos, como um filtro que destaca determinadas propriedades, como, por exemplo, as bordas de uma figura.

5.8. Bloco *FeatureMap* e *ConvLayer*

A *feature map* é a saída gerada pela convolução detalhada no item anterior. A aplicação de cada filtro (*kernel*) produz uma resposta espacial sobre a entrada e, quando vários filtros são aplicados, o resultado passa a representar diferentes características extraídas da imagem, como bordas, texturas ou padrões locais.

Em uma imagem RGB, por exemplo, cada canal pode ser processado por filtros específicos, e os resultados são combinados para formar a *feature map* final. A camada *ConvLayer* organiza esses filtros em conjunto, permitindo extrair múltiplas características ao mesmo tempo e manter a relação entre canais e profundidade da representação.

Em termos práticos, a *ConvLayer* funciona como a estrutura que empilha filtros para transformar a entrada em uma nova representação mais rica, pronta para ser processada pelas etapas seguintes da CNN.

5.9. Conceitos aplicados ao projeto

A documentação do bloco CNN reúne os seguintes conceitos fundamentais:

- `*DotProduct*`: operação de produto interno e soma acumulada entre entradas e pesos.
- `*Convolution*`: operação de sliding window sobre a imagem para geração da feature map.
- `*FeatureMap*`: combinação dos resultados parciais dos canais em uma saída final.
- `*ConvLayer*`: conjunto de *feature maps* com múltiplos filtros e múltiplos canais.
- `*Neuron*`: bloco equivalente a um neurônio com multiplicação, soma e função de ativação *(Este bloco é passível de ser utilizado como uma possibilidade de implementação de etapas de clock-gating para redução de consumo por propagação infinita de clock em IDLE).*
- `*FCLayer*`: camada totalmente conectada, com todos os neurônios conectados às entradas.
- `*MaxPooling*`: seleção do maior valor em regiões 2x2 para reduzir a dimensionalidade.
- `*ReLU*`: ativação que anula valores negativos e mantém positivos.
- `*Quantizer*`: conversão de valores para o formato de ponto fixo utilizado.
- `*Flatten*`: reorganização das *feature maps* em vetor unidimensional.
- `*CNN*`: top module que monta todos os blocos em sequência.

6. Síntese

Os relatórios de síntese disponibilizados em `*synth_org/top/reports/asap7/top/base/*` contêm representações visuais do layout, congestionamento, roteamento e análise de *timing*. A seguir, alguns exemplos dos arquivos `*.webp*` gerados:

6.1. *Constraints* do projeto:

Para a síntese deste *ASIC* foi utilizado um *SDC* simples com foco em um clock de 100Mhz:

```
current_design CNN

# ==============================================================
# Clock
# ==============================================================
set clk_name core_clk set clk_period 10000 set clk_io_pct 0.20 set clk_port [get_ports clk]
create_clock \
-name $clk_name \
 -period clkperiod clk_port
# ==============================================================
# Input constraints
# ==============================================================
set non_clock_inputs [all_inputs -no_clocks] set_input_delay \
 [expr $clk_period * $clk_io_pct] \
-clock clkname non_clock_inputs
# ==============================================================
 # Output constraints
# ==============================================================

set_output_delay \
[expr $clk_period * $clk_io_pct] \
-clock $clk_name \
[all_outputs]
```

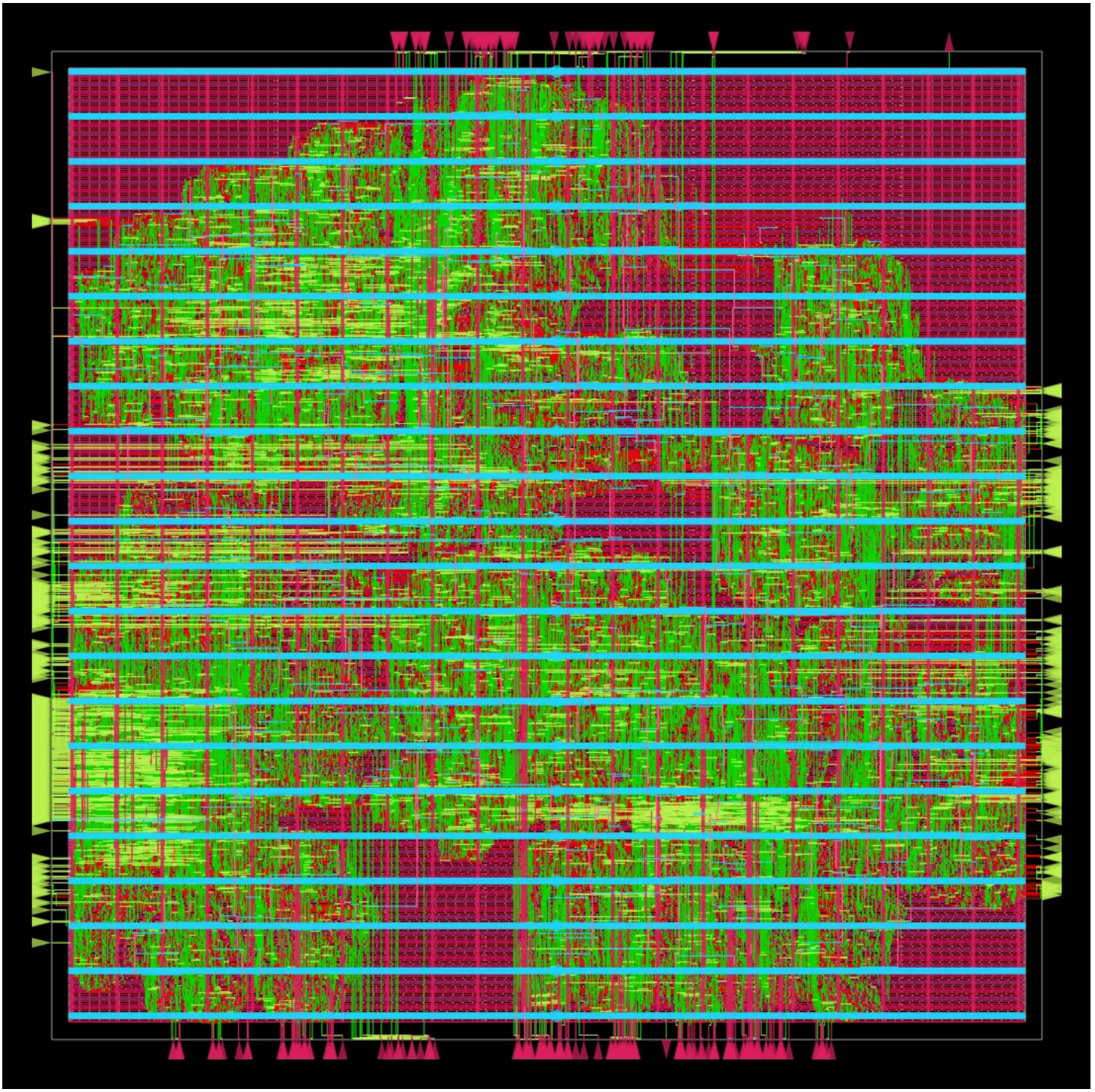
6.2. Visão Geral do *Layout* da implementação

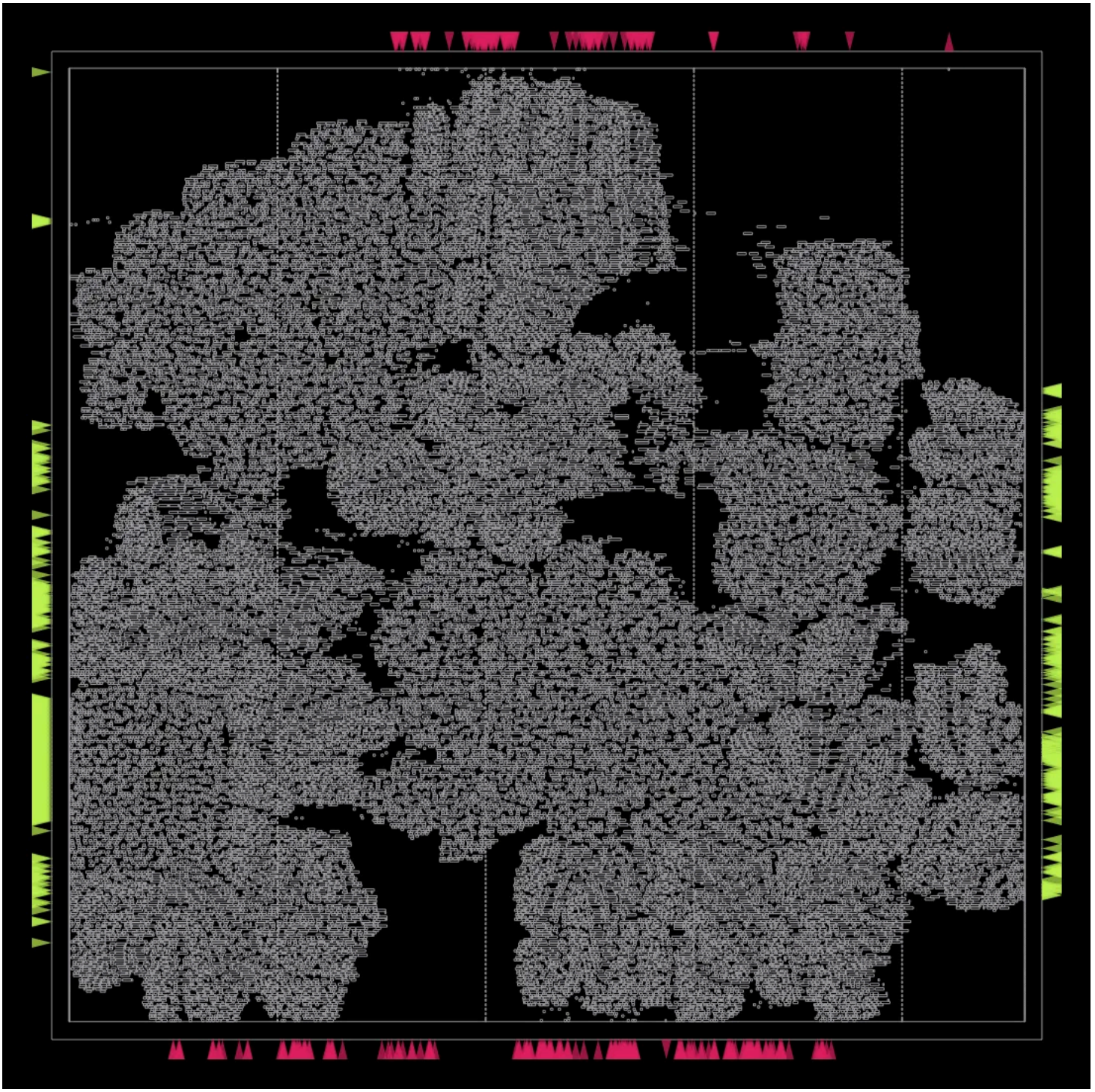
O layout físico do chip foi gerado, demonstrando a disposição das células padrão e o roteamento das interconexões. As imagens fornecidas mostram:

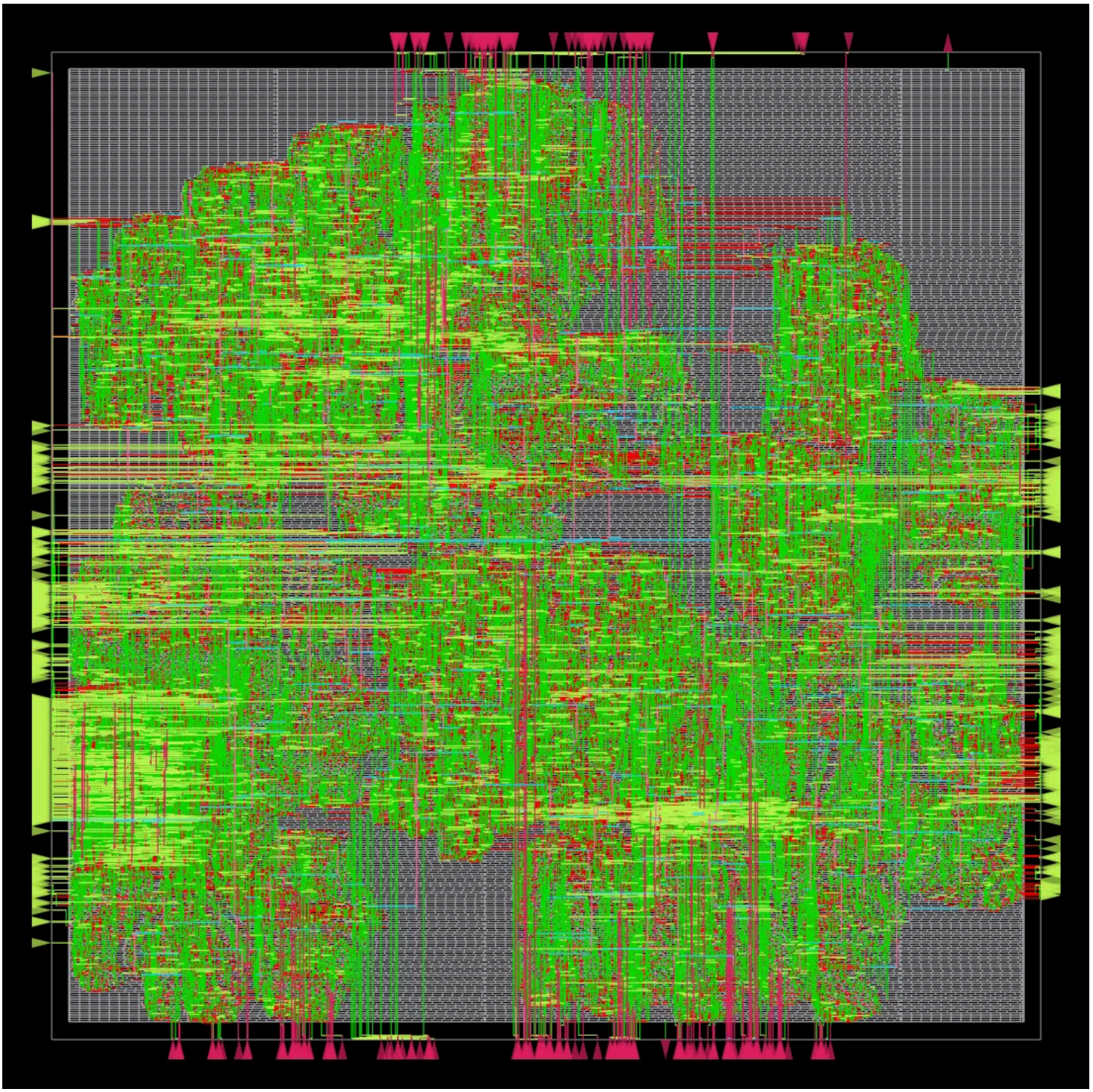
- **Layout de geral do layout e da implementação:** Vista em corte do *floorplane* com a camada de *PDN*;
- **Layout de posicionamento e roteamento:** Vista em corte do *floorplane* que demonstra os roteamentos e deixando explícito onde há células FILLER;

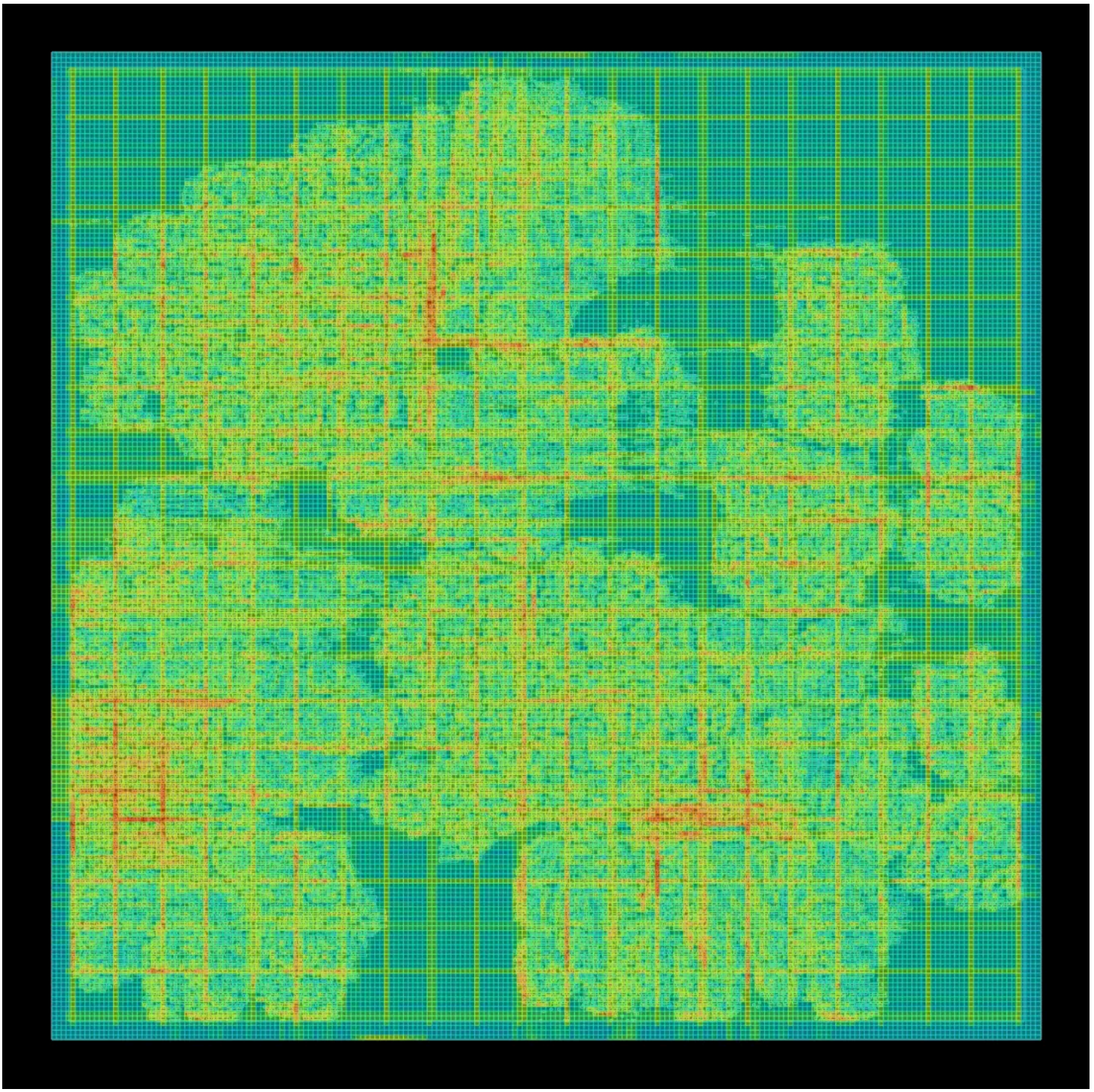
- **Layout de congestionamento e potência:** Vista que demonstra os caminho críticos à observar;
- **Layout de clock e timing:** Vista em corte do *floorplane* demonstrando a CTS.

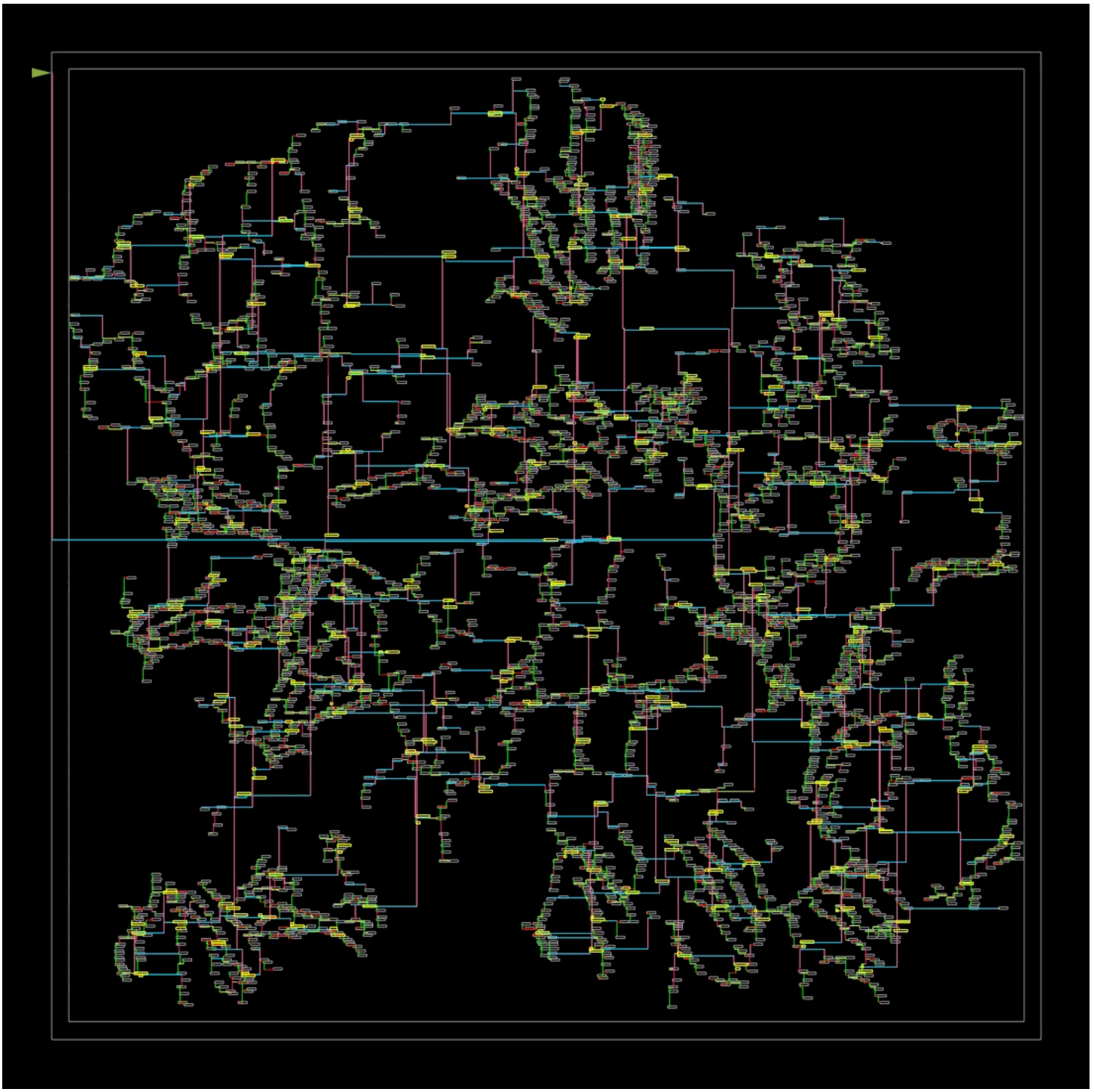
Visão geral do layout e da implementação

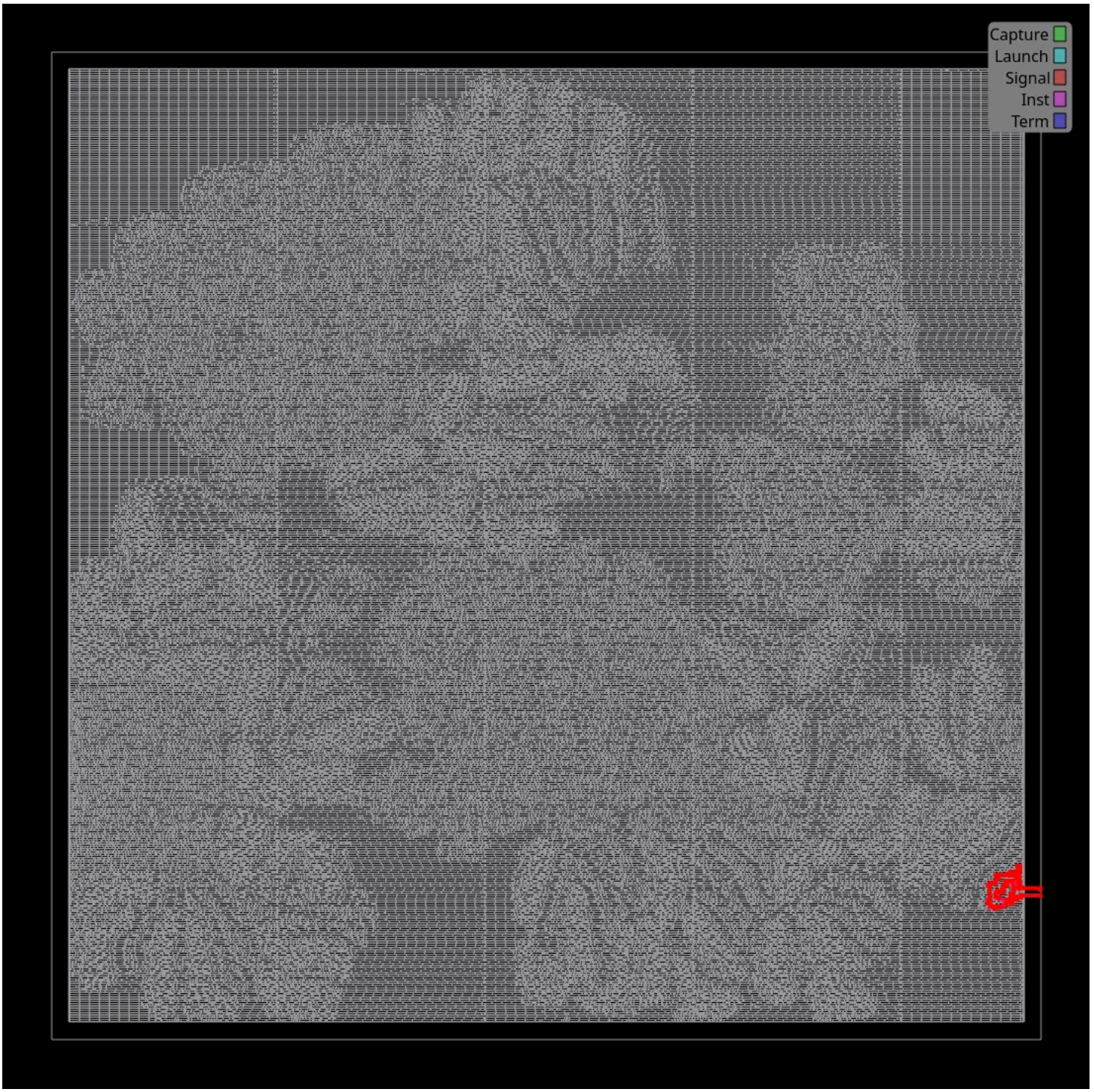












6.3. Contagem de Células (*Cell Count*)

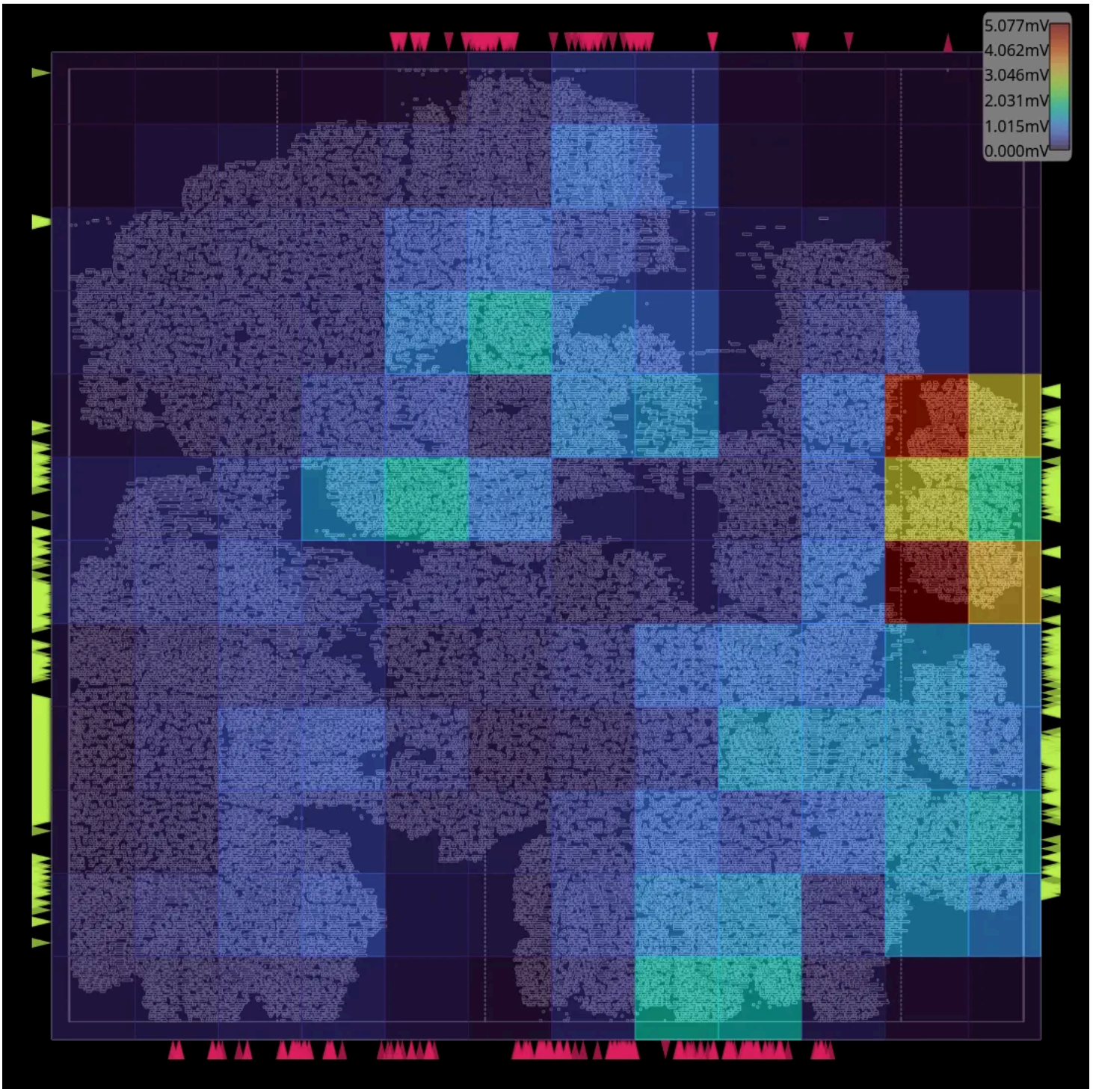
O arquivo `*synth_org/top/reports/asap7/top/base/synth_stat.txt*` registra as células efetivamente inseridas no design após a síntese lógica. A tabela abaixo resume as principais células e sua contagem/área no módulo `*CNN*`.

Tipo de Célula	Contagem	Área (µm²)
AND2X2_ASAP7_75t_R	3.039	2.852
AND2X4_ASAP7_75t_R	15	2.187
AND3X1_ASAP7_75t_R	1.183	103.489
AND4X1_ASAP7_75t_R	260	26.536
AND5X1_ASAP7_75t_R	109	12.714
AOI21X2_ASAP7_75t_R	159	37.092
AOI21X1_ASAP7_75t_R	1.205	105.413
AOI22X1_ASAP7_75t_R	197	28.723
AOI22X2_ASAP7_75t_R	2	321
AOI22X2_ASAP7_75t_R	22	3.849
AOI22X1_ASAP7_75t_R	81	10.629
AOI22X2_ASAP7_75t_R	3	437
AOI31X2_ASAP7_75t_R	86	20.062
AOI32X1_ASAP7_75t_R	103	12.014
AOI33X2_ASAP7_75t_R	13	1.895
AOI211X1_ASAP7_75t_R	259	45.315
AOI21X1_ASAP7_75t_R	339	39.541
AOI221X1_ASAP7_75t_R	87	17.758
AOI22X1_ASAP7_75t_R	76	11.081
BUFX2_ASAP7_75t_R	1.242	90.542
BUFX3_ASAP7_75t_R	65	5.686
BUFX4f_ASAP7_75t_R	5	583
BUFX6f_ASAP7_75t_R	4	583
DFFHQNX1_ASAP7_75t_R	3.352	977.443
FAX1_ASAP7_75t_R	2.728	556.839
HAXp5_ASAP7_75t_R	4.432	581.567
INVX1_ASAP7_75t_R	12.883	563.502
Sub total - células principais	31.949	3.258.653
Outras células	15.521	2.001.347
Total Geral	**47.470**	**5.260.000**

6.4. Análise de potência e *report_power*

O relatório gerado pelo OpenROAD inclui o resumo do `*report_power*` em `*synth_org/top/reports/asap7/top/base/6_finish.rpt*`. Esse trecho resume a distribuição da potência por grupo lógico do circuito e é uma referência importante para avaliar a eficiência energética da CNN sintetizada.

Grupo	**Internal (W)**	***Switching* (W)**	***Leakage* (W)**	**Total (W)**	**Participação**
Sequential	1.13e-03	7.18e-05	5.11e-07	1.21e-03	11.4%
Combinational	4.51e-03	4.28e-03	3.07e-06	8.80e-03	83.4%
Clock	3.42e-04	2.04e-04	1.23e-07	5.46e-04	5.2%
Macro	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Total	5.98e-03	4.56e-03	3.71e-06	1.05e-02	100.0%



A imagem acima complementa o relatório de potência e mostra a análise de distribuição de tensão e quedas no grid de alimentação, o que é relevante para verificar a robustez da implementação física.

6.5. Trecho de STA do OpenROAD

O processo de síntese também executa análise de timing no final da etapa de implementação. O trecho abaixo foi extraído do relatório final `*synth_org/top/reports/asap7/top/base/6_finish.rpt*` e mostra a saída típica da STA do *OpenROAD*:

```
=========================================================================
finish report_wns
-------------------------------------------------------------------------
wns max 0.00

=========================================================================
finish report_worst_slack
-------------------------------------------------------------------------
worst slack max 5497.88

=========================================================================
finish report_checks -path_delay min
-------------------------------------------------------------------------
Startpoint:
ConvLayer2.gen_feature_maps[1].filter.gen_channels[0].filters.convolve.soma[0][146]$_DFF_
P_

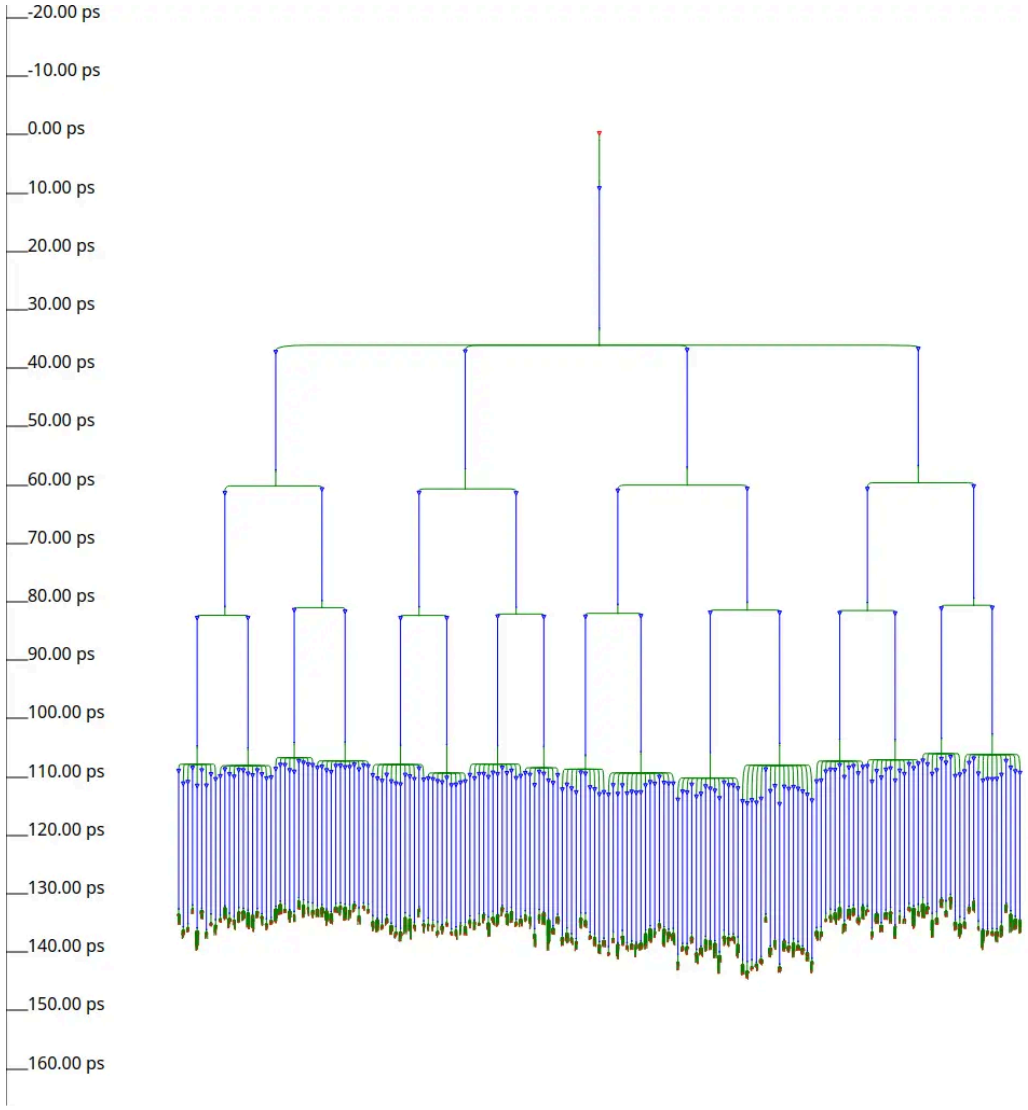
Endpoint:
ConvLayer2.gen_feature_maps[1].filter.gen_channels[0].filters.convolve.soma[0][266]$_DFF_
P_Path Group: core_clk
Path Type: min
...
                         149.37   data required time
                        -167.08   data arrival time
-------------------------------------------------------------------------
                         17.71   slack (MET)

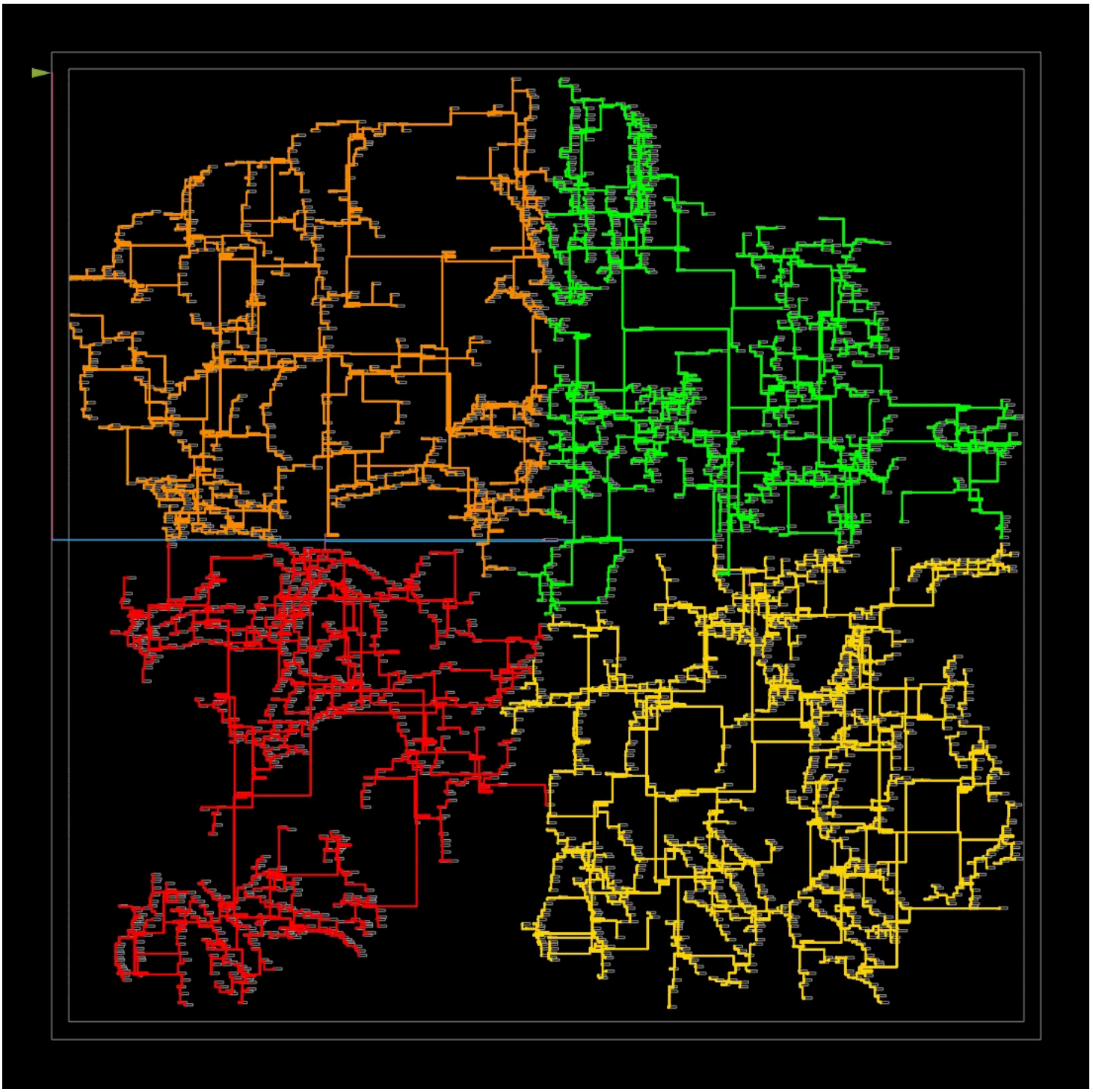
=========================================================================
finish setup_violation_count
-------------------------------------------------------------------------
setup violation count 0

=========================================================================
finish hold_violation_count
-------------------------------------------------------------------------
hold violation count 0
```

Esse trecho indica que o relatório final não reporta violações de *setup* nem de *hold*. O `*worst slack*` do caminho crítico permanece positivo (`5497.88`), e os contadores de violação de timing aparecem como `0`, o que demonstra que a síntese foi concluída sem violações de timing no conjunto de relatórios avaliados.

6.6. Relatórios de CTS e layout associado





Esses artefatos são fundamentais para avaliar a qualidade da implementação física da CNN, incluindo:

- posicionamento dos módulos;
- distribuição dos clocks;
- roteamento final;
- congestionamento de interconexões;
- análise de potência e queda de tensão;
- caminho crítico e timing.

7. Verificação

Para que fosse possível validar o comportamento do bloco foi adotada a estratégia de testar os principais blocos separadamente e ao final verificar o topo, todas as bancadas podem ser consultadas na pasta UVM do repositório, aqui iremos adentrar no detalhamento apenas da bancada de topo, esta foi construída com a seguinte estrutura:

```
├── req_verif_CNN_top.md
├── testbench.sv
├── UVM_agent.sv
├── UVM_config.sv
├── UVM_coverage.sv
├── UVM_driver.sv
├── UVM_env.sv
├── UVM_monitor.sv
├── UVM_reference.sv
├── UVM_scoreboard.sv
├── UVM_seq_item.sv
├── UVM_sequencer.sv
├── UVM_sequence.sv
├── UVM_tb_params.svh
└── UVM_test.sv
```

7.1. Estratégia de estímulos

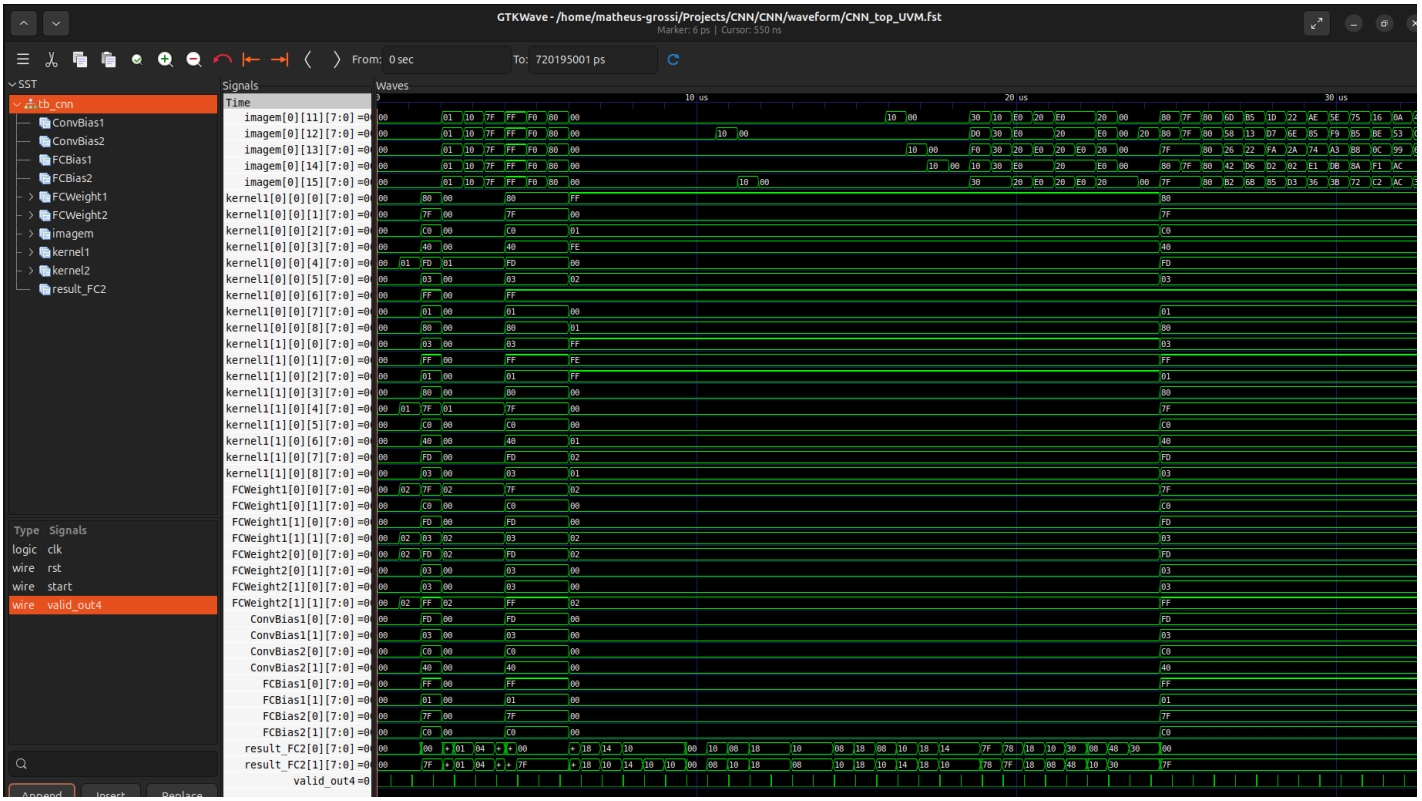
Para verificação deste bloco foi seguida a regra padrão de caixa preta e nosso plano de testes seguiu a sequência estratégia:

1. Criamos um banco de imagens pseudoaleatórias de 0 a 21
2. Aplicar e liberar o reset inicial de acordo com a semântica do RTL.
3. Configurar os 16 pixels, kernels, pesos FC e todos os *bias*.
4. Manter entradas estáveis antes do espaço de amostragem de `start`.
5. Aplicar `start` por um ciclo de clock em uma transação isolada.
6. Manter imagem, pesos e *bias* estáveis até a conclusão da operação (campanha funcional inicial).
7. Registrar os eventos válidos internos e externos, bem como o vetor final.
8. Comparar com o Golden Model e encerrar a transação antes de iniciar a próxima.

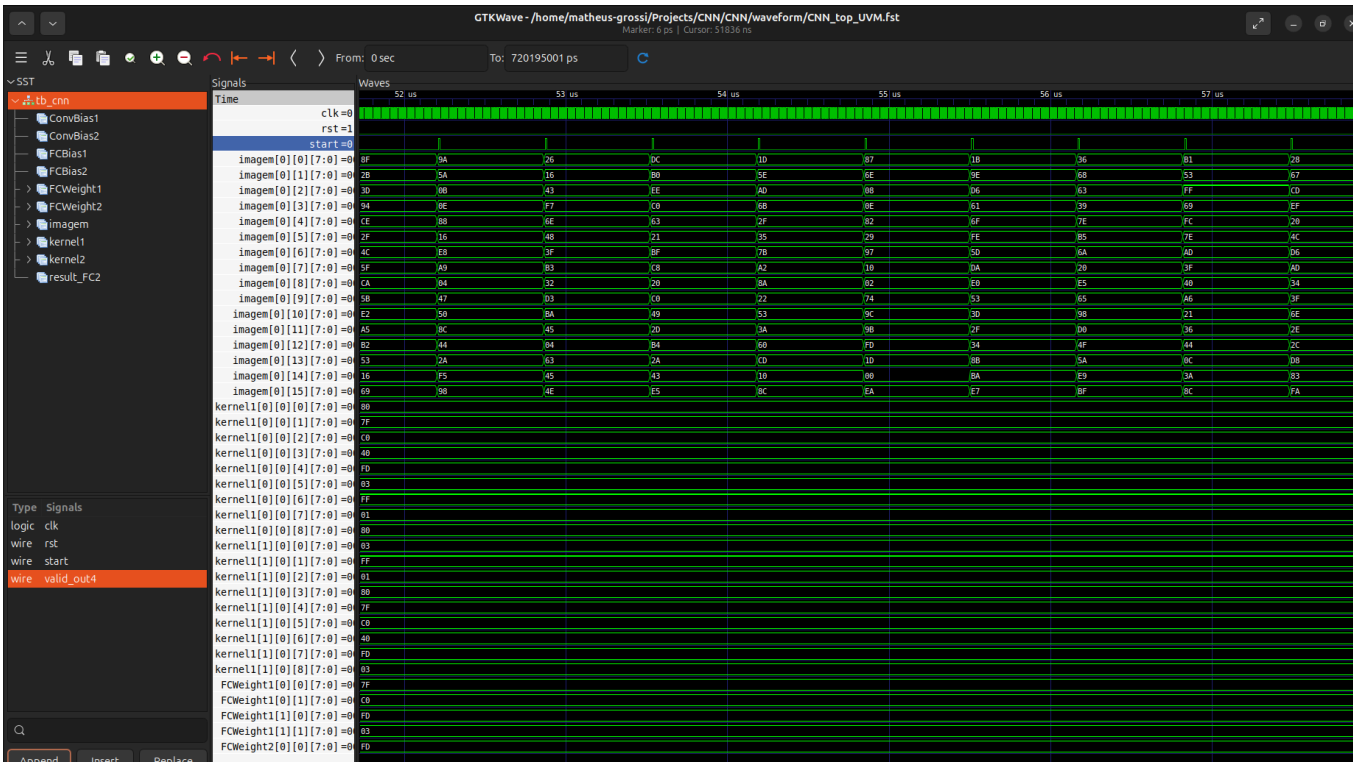
Um modelo de referência calcula os resultados sem consultar os valores das saídas do DUT para descobrir valores esperados.

7.2. Waveforms da simulação

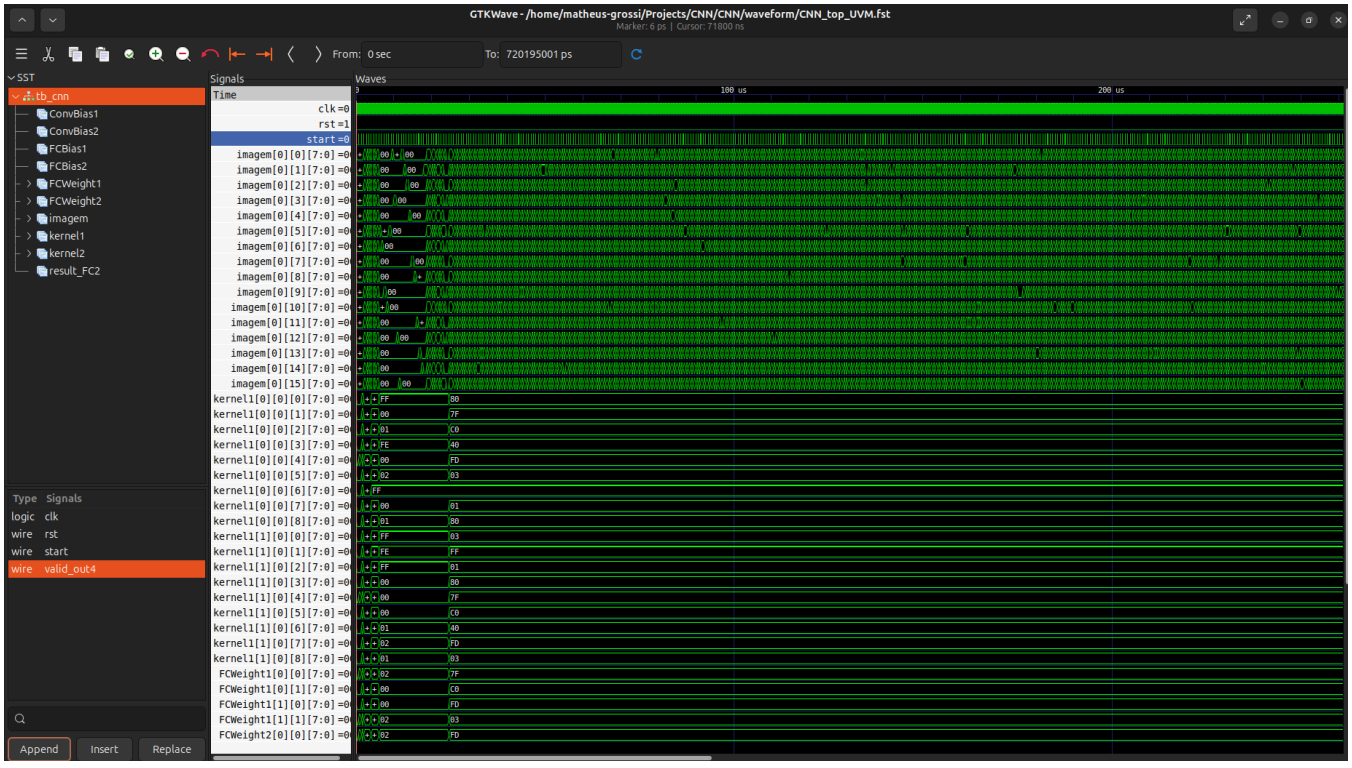
O waveform do topo pode ser obtido na pasta da [main]



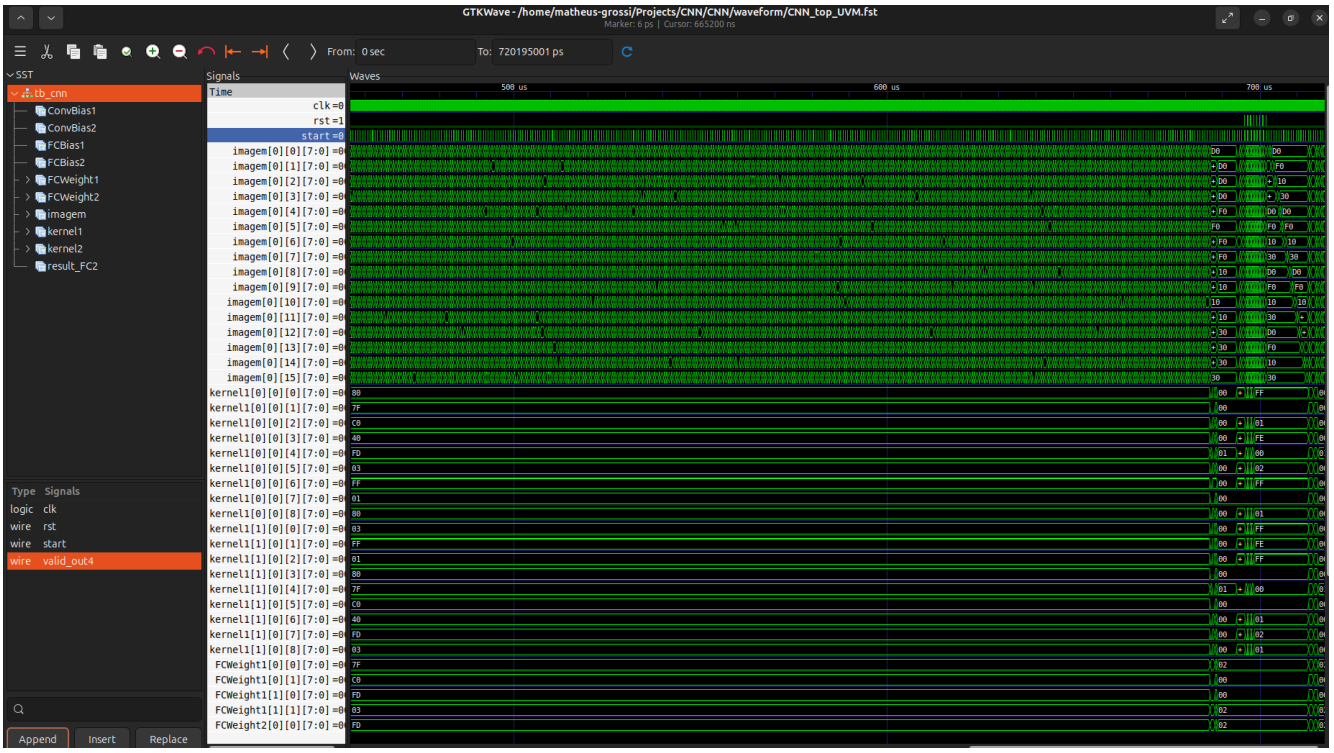
Recorte 1



Recorte 2



Recorte 3



Recorte 4



Recorte 5

7.3. Relatório da taxa de cobertura da verificação

LCOV - code coverage report

			Coverage	Total	Hit
Current view:	top level	**Lines:**	100.0 %	75	75
Test:	CNN_top UVM - cobertura RTL de src (linhas e ramos)	**Functions:**	-	0	0
Test Date:	2026-10-25 15:09:45	**Branches:**	94.3 %	211	199
Legend:	Rating: low: < 75 % medium: >= 75 % high: >= 90 %				

Directory	Line Coverage			Branch Coverage		
	Rate	Total	Hit	Rate	Total	Hit
src/	100.0 %	13	13	98.3 %	58	57
src/Convolution/	100.0 %	39	39	96.9 %	96	93
src/DotProduct/	100.0 %	22	22	84.9 %	53	45
src/ReLU/	100.0 %	1	1	100.0 %	4	4

Note: 'Function Coverage' columns elided as function owner is not identified.

Generated by: LCOV version 2.0-1

O relatório demonstra que conseguimos passar por todas as linhas de código e atingimos uma taxa de cobertura funcional de 94.3%.

7.4. Análise de resultados

Totalizam 1.100 casos randômicos, coordenados de teste usando essa metodologia e encontramos alguns problemas que serão detalhados abaixo:

1.083 PASS
2 FAIL
3 abortadas por reset
12 INCOMPLETE, do cenário exploratório IMG-19

7.5. Casos de falha/Incomplete

As duas falhas foram divergências nos resultados: no IMG-00, com imagem zerada e W03, esperávamos [127,0] e recebemos [0,127]; no IMG-16, com ConvBias2[0] = -45, esperávamos [0,12] e recebemos [127,12]. Ambas responderam em 37 ciclos.
Os 12 incompletos pertencem ao caso da IMG-19: foram enviados três starts para cada intervalo de 2, 3, 4 e 8 clocks. e como a interface não possui um sisteminha hs de ready/busy nem um contrato definido para sobreposição de imagens, essas transações foram classificadas como exploratórias, sem aprovação ou reprovação funcional.

7.6. Análise final da bancada

Há dois apontamentos importantes, a bancada excita apenas 84.9% de *DotProduct*, o que não necessariamente implica que haja janelas de desconhecimento comportamental pois o bloco *Dot* foi alvo de uma verificação isolada pra ele que atingiu taxas significativas de verificação e podem ser encontradas em Verificação DOT.

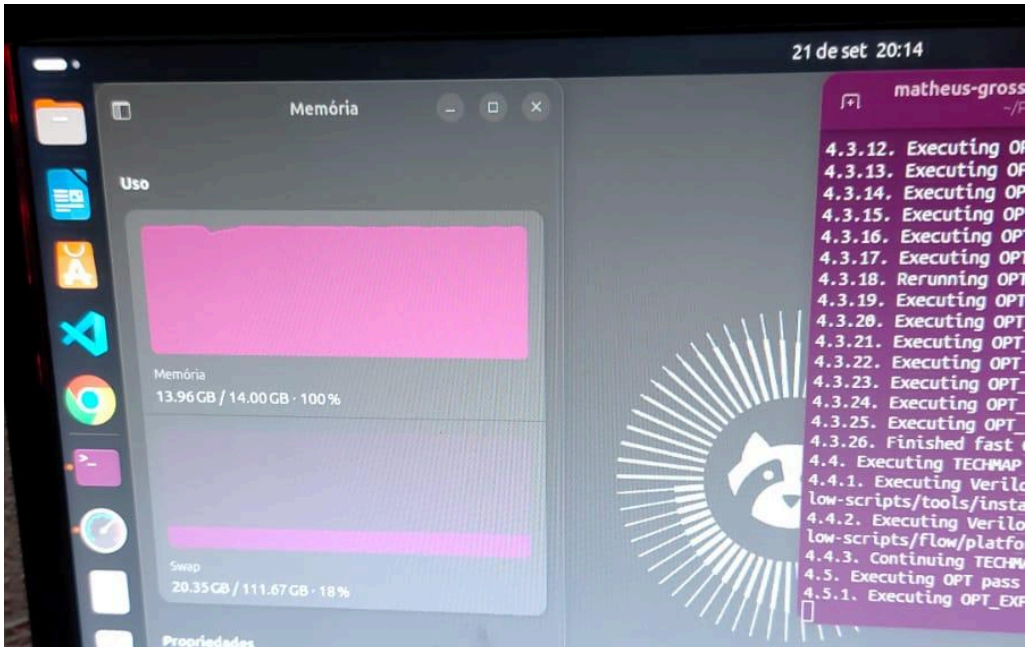
O bloco possui os defeitos funcionais apontados e em uma aplicação real não estaria apto à prosseguir para a implementação final, sendo necessário passar por uma revisão extensiva da equipe de desenvolvimento a fim de obter um ASIC funcional.

O relatório de verificação pode ser encontrado no documento Verif_top.

8. Limitações no desenvolvimento e solução de contorno adotadas

Os três integrantes do grupo não estão residindo nas proximidades do novo polo da HBR, Matheus e Felipe, inclusive, sequer moram no mesmo município. Devido à essas questões, focamos no fluxo de projeto Open-Source para executar toda a rotina de trabalho.

A síntese do projeto foi feita utilizando-se do nó tecnológico de 7nm ***ASAP7*** que é um *PDK* genérico desenvolvido pela Universidade do Arizona. Como nosso projeto é extremamente complexo em nível lógico tivemos severos problemas para conseguir fazê-lo até o final, as etapas de roteamento com *effort* padrão demandam quantidades exorbitantes de memória RAM como pode ser visto na imagem abaixo o qual o *OpenRoad* consumiu meus 16gb e precisou de 20gb de RAM em Swap naquele momento (No auge foram 50gb):



Para compilação trabalhamos com ***Icarus/Verilator*** sendo que os *testbenchs* simples eram feitos no *Icarus* e as bancadas *UVM* no *Verilator*.

Devido a problemas de compatibilidade e poderio computacional foi impossível rodar a simulação com anotação *sdf* do projeto e infelizmente também não contávamos com acesso remoto ao servidor da HBR e portanto essa etapa não pode ser feita.

9. Scripts

Todo o fluxo de automação de processos foi pensado em uma estrutura para *Makefile/Bash*, os *.sh* apenas chamam os *Makes* que por sua vez executam as funções extensivas, todos estes estão salvos na *branch* final do projeto e podem ser acessados.

10. Conclusão

O desenvolvimento do projeto possibilitou a fixação dos conceitos aprendidos ao longo do Programa CI Digital, tendo sido desenvolvido de forma estruturada. Este repositório reúne o conjunto de blocos que modelam uma CNN simples, desde a implementação e verificação do comportamento em diferentes etapas do fluxo de design, processos de síntese e análise de resultados, visando uma abordagem adequada para fins acadêmicos e extensão em futuras versões da arquitetura.

11. Referências

- O projeto está hospedado no repositório GitHub: https://github.com/GRG-Semicondutores/CNN/tree/main
- Aulas do programa CI Digital

12. Anexos

Explicação completa CNN.sv

O módulo CNN.sv é o top module, ou seja, serve para instanciar todos os outros módulos dentro dele, e não implementa nada por si mesmo.

Todos os parâmetros explicados

- ***DATA_WIDTH***: largura do barramento de dados, ou seja, quantos bits cada número na CNN possui. Ex. 8 bits, cada peso, entrada e qualquer outro dado tem uma representação que vai de 00000000 a 11111111.
- ***IMG_SIZE_1***: Como a CNN tipicamente trabalha com imagens, esse é o tamanho do lado da imagem, em pixels. Para *IMG_SIZE_1* = 4, teríamos então uma imagem 4x4.
- ***KERNEL_SIZE***: O *kernel* é uma "mini imagem" que faz convolução com a imagem maior. Não é uma imagem propriamente dita, pois seus "pixels" representam pesos, ao invés de cores reais, mas a ideia de um quadrado NxN se permanece aqui. Para *KERNEL_SIZE* = 3, temos um *kernel* 3x3.
- ***N_CHANNELS***: Cada *pixel* de uma imagem convencional tem 3 valores: R, G, B, que representam uma cor baseada na mistura das cores de *Red*, *Green* e *Blue*. Cada um desses valores vai de 0 a 255 (ou seja, os 8 bits do *DATA_WIDTH* vão bem aqui). Para que possa representar uma imagem, então, eu preciso ter "3 imagens diferentes", a imagem do *Red*, do *Green* e do *Blue*. Quando juntas, formam a imagem original. A estas "imagens diferentes", chamamos de canais. N_CHANNELS representa, portanto, o número de canais da imagem atual.
- ***N_FILTERS_LAYER_1 / _2***: cada *kernel* faz uma convolução com cada canal da imagem, e depois soma pixel a pixel dos canais. Um *kernel*, possuindo seus pesos, se especializa em pegar uma característica (feature) da imagem. Porém, nos trabalhos de CNNs, observou-se que pegar mais de uma feature da imagem naquela mesma camada é benéfico para o resultado final. Então, coloca-se vários kernels para extrair várias features da imagem, cada kernel convoluindo com cada canal. A estes vários kernels se dá o nome de filtros. Temos então, o número de filtros da camada 1 e o número de filtros da camada 2.
  Ex. Uma imagem inicialmente com 3 canais RGB é convoluída por 5 filtros. Cada um desses filtros (kernels) convolui com o canal R, o canal G e o canal B. Após a convolução, os pixels resultantes do R, do G e do B, são somados *pixel* a *pixel*, ou seja: *pixel_1 = pixel_1_R + pixel_1_G + pixel_1_B*. *Pixel_n = pixel_n_R + pixel_n_G + pixel_n_B*.
  Isso significa que de uma imagem com 3 canais, após a convolução com um filtro, a "imagem" resultante (que chamamos agora de feature, pois não se assemelha mais com uma imagem real, mas uma característica da imagem) tem apenas 1 canal.
  Como 5 filtros fazem o mesmo processo, acabamos tendo 5 features diferentes. Cada uma dessas features são na verdade, o número de canais da feature resultante. Ou seja, o número de canais resultantes = número de filtros aplicados na feature de entrada.

- ***PADDING_1 / 2***: às vezes, a convolução te dá um número ímpar de pixels NxN, o que dificulta em outros processos. Por outras vezes, pode-se perceber que adicionar o padding simplesmente melhora o resultado da convolução, aumentando a precisão da saída. O padding é o preenchimento com 0s ao redor de toda a imagem. Cria, portanto, 2 linhas e 2 colunas adicionais. Utilizamos aqui para fins de compatibilidade com outros blocos (questão do ímpar explicado anteriormente).
- ***N_NEURONS_1 / 2***: Número de neurônios na primeira e segunda camadas de neurônios. Os neurônios são o elemento básico das redes neurais. Possuem um peso para cada entrada, multiplicam peso_n x entrada_n para todas as entradas e pesos, e soma os resultados. Sua saída pode conter uma função que adeque a saída para um resultado mais desejável. Pixels de imagens nada mais são que entradas, e *kernels* nada mais são que pesos "reutilizáveis", que fazem uma varredura nas entradas (ao invés de cada entrada ter seu peso, o kernel age como um conjunto de pesos que é compartilhado para todas as entradas). As camadas de convolução, portanto, são nada mais que uma arquitetura diferenciada de uma rede neural comum.
- ***SHIFT_1/2/3/4***: O shift serve para REALINHAR a escala fracionária escolhida para as camadas trabalharem. Ex: inputs em Q1.7 e weights em Q1.7. A multiplicação entre eles vai gerar um Q2.14. A escala que a outra camada espera também é Q1.7. Logo, espera-se que o shift elimine 7 bits fracionários da escala resultante para voltar à escala desejada. Então a conta do *shift* é *num_de_bits_frac_input + num_de_bits_frac_weights - num_de_bits_frac_output*. Isso gera Q2.7 (9 bits). Como o registro de saída só reserva *SIZE_OUT bits* (formato Q1.7, 8 bits), qualquer valor cujo resultado real precise do segundo bit inteiro será saturado no máximo/mínimo de Q1.7.

Parâmetros locais

Não são definidos pelo usuário, mas derivados dos parâmetros que são:

- ***N_KERNEL***: Número de pesos totais do *kernel*, fazendo-se N * N, com N sendo o *KERNEL_SIZE*.
- ***N_PIXELS***: Idem, para os pixels da imagem/*feature*
- ***OUT_SIZE_1/2***: a convolução entre o kernel e a feature gera uma feature menor. Essa feature menor tem seu tamanho determinado pela fórmula:
  Tamanho da feature – tamanho do kernel + 1.
  Se adicionarmos padding na feature, vira:
  *Tamanho da feature + 2 * padding – tamanho do kernel + 1.*
- ***N_OUT***: Mesma ideia de *N_KERNEL* e *N_PIXELS*, calcula o total de *pixels* da feature resultante, pegando o lado de *OUT_SIZE* e multiplicando por ele mesmo.
- ***N_OUT_POOLED***: O bloco *MaxPooling* performa uma operação de max pooling. Essa operação seleciona um quadrado de 4 pixels (2x2) na imagem, e escolhe dentre esses 4, o maior, e elimina os 3 restantes. Faz isso para cada conjunto de 2x2 na imagem. Este processo resulta em uma imagem com 4 vezes menos pixels. Portanto, *N_OUT_POOLED = N_OUT / 4*.
- ***IMG_SIZE_2***: Tamanho do lado N da "imagem" da segunda camada. Aqui, na verdade, já é chamada de feature, pois é uma característica da imagem original, e não representa muito bem uma imagem com seus pixels. *N_OUT* seria essa própria feature, não fosse pela

operação de pooling feita posteriormente. Por conta do pooling, o número de pixels é reduzido por 4, enquanto o tamanho do lado é reduzido por 2. Portanto, *IMG_SIZE_2* = *N_OUT* / 2.

- ***N_CHANNELS_2***: Como dito anteriormente, o número de canais de uma feature é igual ao número de filtros que a feature passada convoluiu. Portanto, o número de canais da segunda camada (*N_CHANNELS_2*) é o número de filtros da primeira camada (*N_FILTERS_LAYER_1*).
- ***OUT_SIZE_2***: Mesma coisa de *OUT_SIZE_1*, apenas para segunda camada.
- ***N_OUT_2, N_OUT_POOLED_2***: Mesma coisa da camada anterior, apenas para segunda camada.
- ***N_FLAT***: A camada de neurônios não entende quando as entradas estão em formato (NxN) * *N_FILTERS*, ou seja, um conjunto de N_FILTERS features de tamanho NxN. As entradas da camada de neurônios devem ser unidimensionais, enquanto aqui estamos apresentando 3 dimensões (N, *N* e *N_FILTERS*). Este parâmetro apenas calcula quantas entradas terão N * N * N_FILTERS.
- ***ACC_WITDH***: Multiplicação de números que possuem *DATA_WITDH* de tamanho resultam em um valor de até *DATA_WITDH* * 2 de tamanho. Como existem somas sucessivas além das multiplicações, é capaz de ter um tamanho ainda maior, determinado pelo *log2* dos pesos dos *kernels* multiplicado pelo número de canais.

INPUTS e OUTPUTS

- ***Clk***, ***rst***: autoexplicativos.
- ***Start***: um bit de pulso, que representa o início da operação da CNN. Esse pulso deve se propagar para cada bloco em cada camada, para as operações operarem sincronizadamente.
- ***Imagem***: os *pixels* que representam uma imagem. Cada pixel pode ter um valor máximo de *DATA_WIDTH* (típico 8 bits, 255 int). A imagem tem *N_PIXELS* pixels no total, e *N_CHANNELS* canais no total (típico 3 canais: R, G e B)
- ***Kernel1***, ***kernel2***: os pesos (“pixels”) que representam um *kernel*. Cada peso pode ter um valor máximo de *DATA_WIDTH*. O kernel tem *N_KERNEL* pesos. A quantidade de *kernels* (filtros) é determinada por *N_FILTERS_LAYER_X*. Para fins de compatibilidade com blocos, também é informado quantos canais cada kernel vai convoluir (no caso da imagem inicial com 3 canais, cada kernel convolui com 3 canais). Em *kernel2*, o número de canais é o número de filtros da *layer* anterior, como previamente discutido.
- ***FCWeight1***, ***FCWeight2***: Pesos da camada de neurônios. Cada peso pode ter um valor máximo de *DATA_WIDTH*. Cada neurônio tem seus pesos (portanto, *N_NEURONS* conjuntos de pesos), e o número de pesos por neurônio é igual ao número de entradas (portanto, *N_FLAT* entradas), *N_FLAT* sendo o número de entradas que são todos os canais das últimas features da última camada antes dos neurônios.
- ***ConvBias1***, ***ConvBias2***: Os neurônios, além de fazerem multiplicação dos pesos pelas entradas e soma de todos os resultados, somam também um bias, que é um valor independente de multiplicação. Pense na função afim ax + b. x são as entradas, a são os pesos, e b é o bias, um termo separado para poder “mover a função” livremente. As camadas de convolução não são diferentes, e também precisam de bias, que são estes.
- ***FCBias1***, ***FCBias2***: O bias dos neurônios que foi descrito anteriormente.

- ***Result_FC2***: É o resultado final da CNN, onde idealmente veríamos as 10 classes separadas em probabilidade.
- ***Valid_out4***: é o *bit* de validade que indica que a imagem passou por toda a CNN e chegou na camada final, indicando que o resultado está pronto para ser lido.

Sinais lógicos intermediários

- ***Result1***: resultado da primeira convolução entre a imagem e os *kernels* da primeira camada.
- ***Result1_quantized***: mesmo resultado, apenas quantizado.
- ***Result1_relu***: mesmo resultado, agora passando pela função *relu*, que transforma em 0 qualquer resultado negativo.
- ***Result1_pooled***: mesmo resultado, depois de passar pela relu, agora formatado pela operação de *MaxPooling*, sendo agora o resultado final da operação na camada (virou feature para a próxima camada).
- ***Result2_xxxxxx***: mesma coisa, para a segunda camada.
- ***Result2_flatten***: resultado final da segunda camada, agora, destrinchado em uma única dimensão (ou seja, pegamos a feature NxN, com *N_CHANNELS* canais, e transformamos em um vetor unidimensional de valores, de tamanho N * N **N_CHANNELS*). Serve como entrada para a primeira camada de neurônios.
- ***Result_FC1***: Vetor de saídas resultantes da primeira camada de neurônios, que servem de entrada para a segunda camada de neurônios.
- ***Valid_out1***, ***valid_out2***, ***valid_out3***: bits de válido para que a próxima camada saiba que pode trabalhar em cima daquele valor obtido naquele tempo.

Blocos declarados

Agora, entendidos todos os sinais, a CNN declara todos os blocos modulares, transformando-os em camadas interconectadas, que formam toda a CNN.

- **Primeiro bloco ConvLayer**: Camada convolucional, que vai convoluir *N_FILTERS_LAYER_1 kernels* (filtros) com uma imagem com *N_CHANNELS* canais.
  O resultado é gravado em *result1*. Após isso, o resultado é quantizado (*Quantizer*), passado por uma *ReLU*, e seu tamanho é diminuído por um *MaxPooling*, obtendo o resultado final da primeira camada.
- O **segundo bloco** vem logo depois, com mais uma *ConvLayer*, agora trabalhando em cima da feature que é resultado da camada anterior (*result1_pooled*). Mais uma camada convolucional, que passará por quantização, *ReLU* e *MaxPooling*.
  Repare que a estrutura é a mesma da camada anterior.
- O **terceiro bloco é a camada FC** (camada de neurônios). Primeiramente, o bloco Flatten faz a operação de transformar a última imagem de *N_CHANNELS* canais, NxN *pixels* em um vetor unidimensional de entradas.
  Após isso, é aplicada a primeira camada de neurônios (*FCLayer*), com seus pesos, bias e internamente ela já faz *ReLU*.

- O **quarto bloco é a última camada FC**. A saída da primeira *FCLayer* vira entrada da segunda camada. A segunda camada de neurônios é aplicada, da mesma forma que a primeira, e tendo 10 neurônios na segunda camada, obtém-se 10 saídas, que seriam idealmente as 10 classes que a imagem inicial poderia ser.